# COER UNIVERSITY

## DBMS LAB ASSIGNMENT | LAB SHEET 1: DATA DEFINITION LANGUAGE (DDL)

Task 1: Database & Table Creation

### Question 1.a

*Question: Create a database named Company.*

**Answer:**

```
-- Create a new database named Company
CREATE DATABASE Company;

-- Switch to the newly created database
USE Company;
```

**Explanation:** The CREATE DATABASE statement creates a database logical container named Company in MySQL. The USE statement selects Company as the active database for subsequent operations.

### Question 1.b

*Question: Create a table named Employees with the following structure: EmpID (INT, AUTO_INCREMENT, PRIMARY KEY), FirstName (VARCHAR(50), NOT NULL), LastName (VARCHAR(50)), Department (VARCHAR(50)), Salary (DECIMAL(10,2)), Email (VARCHAR(100), UNIQUE).*

**Answer:**

```
-- Create Employees table with specified column constraints
CREATE TABLE Employees (
    EmpID INT AUTO_INCREMENT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50),
    Department VARCHAR(50),
    Salary DECIMAL(10,2),
    Email VARCHAR(100) UNIQUE
);
```

**Explanation:** This statement defines the Employees schema. EmpID automatically increments for new rows and serves as the primary key. FirstName is mandatory (NOT NULL), and Email enforces uniqueness across records.

## Task 2: DDL Queries on the Employees Table

### Question 2.a

*Question: Describe the structure of the table.*

**Answer:**

```
-- View the structure and details of the Employees table
DESCRIBE Employees;
```

**Explanation:** The DESCRIBE command displays column names, data types, nullability, keys, default values, and auto-increment properties of the table.

### Question 2.b

*Question: Rename the table to Staff.*

**Answer:**

```
-- Rename table Employees to Staff
RENAME TABLE Employees TO Staff;
```

**Explanation:** RENAME TABLE changes the table name from Employees to Staff in the database catalog without altering stored data or structure.

### Question 2.c

*Question: Add a new column PhoneNumber (VARCHAR(15)).*

**Answer:**

```
-- Add PhoneNumber column to Staff table
ALTER TABLE Staff
ADD PhoneNumber VARCHAR(15);
```

**Explanation:** ALTER TABLE ... ADD appends a new column named PhoneNumber to the existing table definition.

### Question 2.d

*Question: Modify the data type of Salary to DECIMAL(12,2).*

**Answer:**

```
-- Modify data type of Salary column
ALTER TABLE Staff
MODIFY Salary DECIMAL(12,2);
```

**Explanation:** ALTER TABLE ... MODIFY changes the column precision of Salary to support numbers up to 12 total digits with 2 decimal places.

### Question 2.e

*Question: Add a DEFAULT value of 'General' to the Department column.*

**Answer:**

```
-- Set default value for Department column
ALTER TABLE Staff
ALTER Department SET DEFAULT 'General';
```

**Explanation:** Setting a DEFAULT constraint ensures that if a record is inserted without a department, MySQL automatically assigns 'General'.

### Question 2.f

*Question: Drop the PhoneNumber column.*

**Answer:**

```
-- Remove PhoneNumber column from Staff table
ALTER TABLE Staff
DROP COLUMN PhoneNumber;
```

**Explanation:** ALTER TABLE ... DROP COLUMN permanently removes the PhoneNumber attribute from the table.

### Question 2.g

*Question: Add a CHECK constraint to ensure Salary is greater than 0.*

**Answer:**

```
-- Add CHECK constraint on Salary column
ALTER TABLE Staff
ADD CONSTRAINT chk_Salary CHECK (Salary > 0);
```

**Explanation:** Adds a domain constraint named chk_Salary to validate that any inserted or modified salary value is strictly greater than 0.

### Question 2.h

*Question: Rename the column FirstName to GivenName.*

**Answer:**

```
-- Rename column FirstName to GivenName
ALTER TABLE Staff
RENAME COLUMN FirstName TO GivenName;
```

**Explanation:** RENAME COLUMN changes the column identifier from FirstName to GivenName while maintaining its data type and constraints.

### Question 2.i

*Question: Drop the chk_Salary constraint.*

**Answer:**

```
-- Drop CHECK constraint from Staff table
ALTER TABLE Staff
DROP CHECK chk_Salary;
```

**Explanation:** Removes the chk_Salary check constraint from the table definition.

### Question 2.j

*Question: Create an index on the Email column.*

**Answer:**

```
-- Create non-unique index on Email column
CREATE INDEX idx_email ON Staff(Email);
```

**Explanation:** Creates a B-tree index named idx_email on the Email column to speed up search and query operations.

### Question 2.k

*Question: Drop the index on the Email column.*

**Answer:**

```
-- Drop index idx_email from Staff table
DROP INDEX idx_email ON Staff;
```

**Explanation:** DROP INDEX deletes the specified index structure from the table.

### Question 2.l

*Question: Truncate the table to remove all rows without affecting the structure.*

**Answer:**

```
-- Truncate Staff table to delete all rows
TRUNCATE TABLE Staff;
```

**Explanation:** TRUNCATE TABLE removes all records from the table quickly and resets auto-increment counters, while keeping the table structure intact.

### Question 2.m

*Question: Drop the Staff table completely.*

**Answer:**

```
-- Permanently drop Staff table
DROP TABLE Staff;
```

**Explanation:** DROP TABLE deletes the table definition, data, constraints, and indexes permanently from the database.

## Task 3: Conceptual Questions

### Question 3.1

*Question: What is the difference between TRUNCATE and DELETE?*

**Answer:**

1. Command Category: TRUNCATE is a DDL command, whereas DELETE is a DML command.
2. Speed & Efficiency: TRUNCATE is faster as it deallocates memory pages directly; DELETE removes rows one by one.
3. WHERE Clause: DELETE supports conditions using WHERE; TRUNCATE removes all rows unconditionally.
4. Auto-increment Reset: TRUNCATE resets AUTO_INCREMENT counters back to 1; DELETE retains the current counter value.

**Explanation:** Understanding the distinction helps in choosing the right command based on performance needs and transaction safety.

### Question 3.2

*Question: Explain the use of the CHECK constraint.*

**Answer:**

The CHECK constraint enforces domain integrity by ensuring that all values entered in a column satisfy a logical boolean condition (e.g., Salary > 0 or Age >= 18). If an insert or update violates the condition, MySQL throws an error and rejects the operation.

**Explanation:** CHECK constraints prevent invalid data entry at the database level.

### Question 3.3

*Question: Why is it important to define a PRIMARY KEY for a table?*

**Answer:**

1. Uniqueness: Guarantees that every row in the table can be uniquely identified.
2. Entity Integrity: Ensures key columns cannot contain NULL values.
3. Performance: MySQL creates a clustered index on the primary key for fast row lookup.
4. Relationships: Acts as a reference key for foreign keys in other tables.

**Explanation:** Primary keys maintain fundamental database integrity and optimize search queries.

### Question 3.4

*Question: How does ALTER TABLE help in modifying an existing table?*

**Answer:**

ALTER TABLE enables structural changes to an existing table without deleting stored data. It supports adding or dropping columns, changing data types, modifying constraints, setting default values, and managing indexes.

**Explanation:** It allows database schemas to evolve over time without losing existing records.

### Question 3.5

*Question: What is the purpose of creating an index in a table?*

**Answer:**

An index builds a search pointer structure (B-tree) on specified columns to significantly accelerate record retrieval during SELECT queries (WHERE, JOIN, ORDER BY), eliminating the need for full table scans.

**Explanation:** Indexes trade small storage space and insert overhead for much faster reading performance.

## Task 4: Exercises

### Question 4.1

*Question: Create a table named Projects (ProjectID INT PRIMARY KEY AUTO_INCREMENT, ProjectName VARCHAR(100) NOT NULL, StartDate DATE, EndDate DATE, Budget DECIMAL(15,2) CHECK (Budget > 0)). Perform at least 5 DDL operations on Projects, and write a query to retrieve column names.*

**Answer:**

```
CREATE TABLE Projects (
    ProjectID INT PRIMARY KEY AUTO_INCREMENT,
    ProjectName VARCHAR(100) NOT NULL,
    StartDate DATE,
    EndDate DATE,
    Budget DECIMAL(15,2) CHECK (Budget > 0)
);

ALTER TABLE Projects ADD Location VARCHAR(100);

ALTER TABLE Projects MODIFY ProjectName VARCHAR(150);

ALTER TABLE Projects RENAME COLUMN Location TO ProjectCity;

CREATE INDEX idx_proj_name ON Projects(ProjectName);

DROP INDEX idx_proj_name ON Projects;

-- Query to retrieve column names
SELECT COLUMN_NAME
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'Projects' AND TABLE_SCHEMA = 'Company';
```

**Explanation:** Creates the Projects table with constraints, executes 5 DDL operations (ADD, MODIFY, RENAME COLUMN, CREATE INDEX, DROP INDEX), and queries system metadata to display table columns.